

SEC-PRJ-7F_25: Triple Factor Authentication for Social Media

Project Report

1. Introduction

Modern social media platforms face sophisticated cyber threats, rendering traditional single-factor authentication obsolete. This project demonstrates a highly secure, enterprise-grade Triple Factor Authentication (3FA) Gateway. The system enforces strict Zero-Trust principles by requiring proof of Knowledge, Possession, and Inherence (Trusted Device) before granting access to a protected JSON Web Token (JWT) session.

2. Architecture Overview

The architecture is designed to be lightweight, stateless, and mathematically secure, utilizing standard cryptographic protocols rather than relying on vulnerable third-party APIs.

- **Frontend:** A single-page application (HTML/CSS/JS) utilizing the native browser **Web Crypto API** for local key generation.
- **Backend:** A Python-based **FastAPI** microservice handling asynchronous cryptographic verification.
- **Database:** An **SQLite** database ([3fa.db](#)) storing user credentials, public keys, and one-time cryptographic challenges.
- **Session Management:** Stateless **JWT (JSON Web Tokens)** signed with the HS256 algorithm.

3. The Authentication Factors

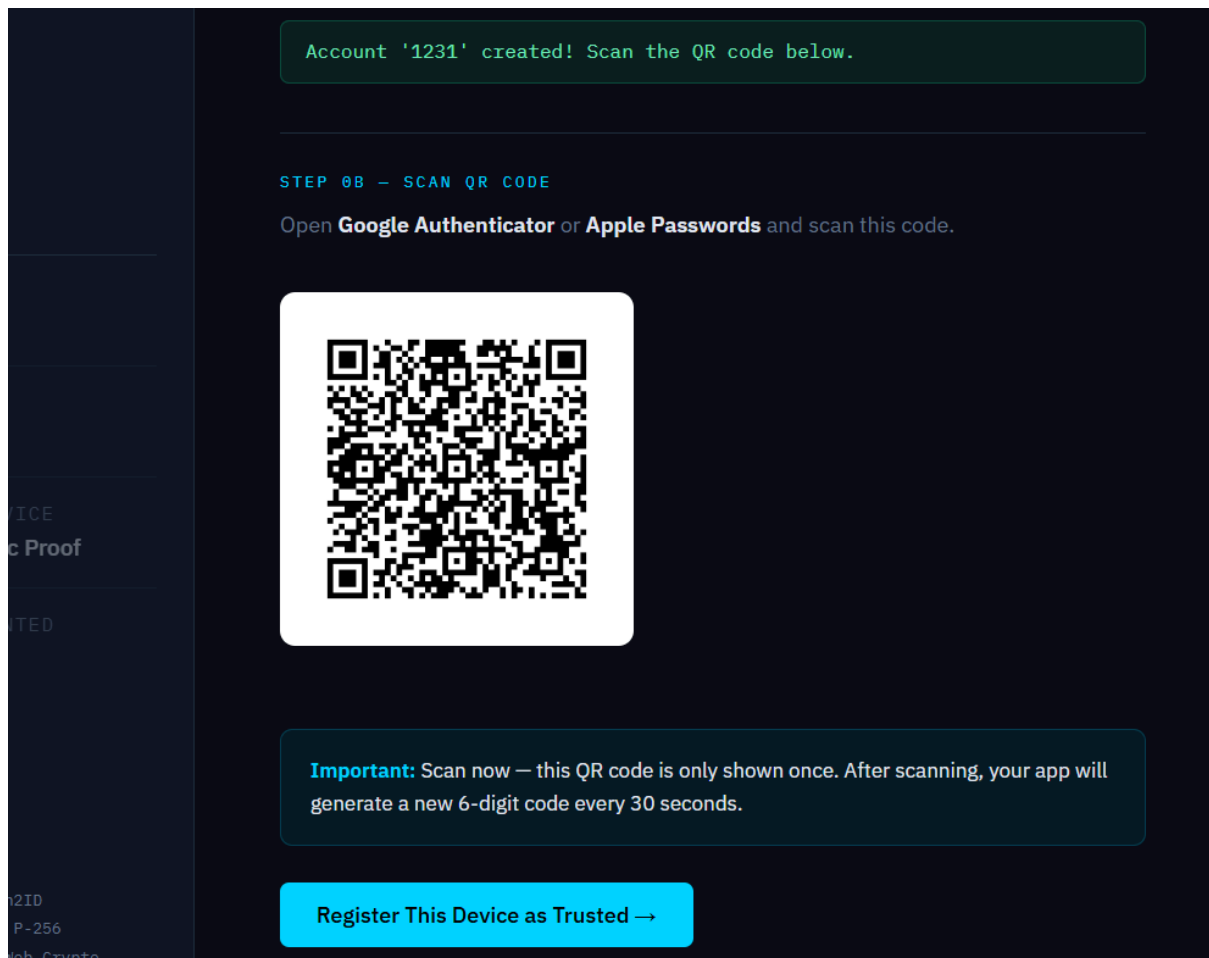
The gateway strictly implements three independent authentication factors to prevent unauthorized access.

Factor 1: Knowledge (The Password)

Upon registration, the user's password is not stored in plaintext. It is hashed using **Argon2ID**, a memory-hard algorithm specifically designed to resist parallelized brute-force attacks from GPUs and ASICs. Upon successful verification, a temporary JWT is issued with a [pending_f2](#) status.

Factor 2: Possession (Time-Based One-Time Password)

The system utilizes the **RFC 6238 TOTP standard**. During registration, the user pairs their account with an authenticator application (e.g., Google Authenticator) via a QR code.



During login, the user must provide a 6-digit code. The backend uses the HMAC-SHA1 algorithm to verify the code against the current 30-second time window. This ensures that even if a password is stolen, the attacker cannot proceed without physical possession of the user's mobile device.

Factor 3: Inherence / Trusted Context (Cryptographic Proof)

The final factor ensures the login request originates from a registered "Trusted Device," neutralizing remote credential-stuffing attacks.

1. **Key Generation:** During registration, the browser's Web Crypto API generates an **ECDSA P-256 Keypair**. The Public Key is sent to the SQLite database, while the Private Key is stored securely within the browser's local storage.
2. **The Challenge:** Upon passing Factor 2, the server generates a random 32-byte string (a cryptographic nonce) and stores it in the **challenges** table.

```

gateway_api > main.py > social_feed
11
12 from fastapi import FastAPI, HTTPException, status
13 from fastapi.middleware.cors import CORSMiddleware
14 from fastapi.responses import JSONResponse
15 from pydantic import BaseModel
16 from dotenv import load_dotenv
17 import os
18
19 load_dotenv()
20
21 from database import init_db, create_user, get_user, user_exists, save_public_key, store_challenge, get_and_clear_challenge
22 from auth import (
23     hash_password, verify_password,

```

	usern...	hashed_password	totp_secret	public_key_jwk
1	Harry	\$argon2id\$v=19\$m=65536,t=3,p=4\$ISLUZR7UMiyeiP...	WCJUAO66YZ4HOOD5B7W5L365FD5L6IXW	{"crv": "P-256", ...}
2	Leonardo	\$argon2id\$v=19\$m=65536,t=3,p=4\$PM/33LgMVzoOnK...	BTMI43CNN7SGHM5MIPHA4PIWBZ4ZJMC	{"crv": "P-256", ...}
3				

3. **The Signature:** The server sends this challenge to the client. The browser uses the hidden ECDSA Private Key to digitally sign the challenge and returns the signature.
4. **Verification & Deletion:** The server verifies the signature using the stored Public Key. If valid, it issues the final JWT and **immediately deletes the challenge from the database**, preventing replay attacks.

SEC-PRJ-7F_25
3FA Gateway

0

SETUP

Registration

1

KNOWLEDGE

Password

2

POSSESSION

TOTP Code

3

TRUSTED DEVICE

Cryptographic Proof

✓

ACCESS GRANTED

JWT + Feed


AUTHENTICATION COMPLETE

All 3 Factors Verified

You have proven: something you **know**, something you **possess**, and that you are on your **trusted device**.

JWT Issued — This signed token is your access credential. It expires in 5 minutes and is only valid with the correct signature.

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJkZW9uYXJkbyIsInN0YXUiOiJmZ1b6x5X2F1dGh1bnRpY2F0ZWQlLCJleHAiOiJlZ3ODI3Njc0MjV9.X1jTyHVIM6_AF80x-70dHI4N1ypAjrufftC1vKx04EWA
```

4. Threat Modeling & Defense Mechanisms

This architecture successfully mitigates several high-profile attack vectors:

- **Server Database Breach:** If an attacker steals the `3fa.db` file, they gain nothing actionable. Passwords are mathematically secured via Argon2ID, and the database only holds Public Keys. The ECDSA Private Keys remain entirely on the users' physical devices (Zero-Knowledge Proof).
- **Network Snooping (Replay Attacks):** Because the cryptographic challenge generated during Factor 3 is a single-use nonce that is deleted upon verification, an

attacker intercepting the network traffic cannot reuse the captured signature for future logins.

- **Phishing & Credential Theft:** If an attacker successfully phishes a user's password and clones their TOTP seed, the login will still be blocked at Factor 3. The attacker's computer does not physically possess the local ECDSA Private Key required to solve the server's challenge.

5. Conclusion

This project successfully implements the requirements for **SEC-PRJ-7F_25**, utilizing OTP, Cryptotools, and JWTs to create a mathematically rigorous defense system. By transforming the user's browser into a hardware token via asymmetric cryptography, the gateway achieves true Triple Factor Authentication without compromising user experience.